

Weather Dashboard Component

jsx

```
import React, { useState, useEffect, useRef } from 'react';

const Weather = () => {
  const [city, setCity] = useState('London');
  const [weatherData, setWeatherData] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const [inputCity, setInputCity] = useState('');

  const abortControllerRef = useRef(null);

  const API_KEY = 'YOUR_API_KEY'; // Replace with your actual Weatherbit API key

  const fetchWeather = async (cityName) => {
    // Abort previous request if exists
    if (abortControllerRef.current) {
      abortControllerRef.current.abort();
    }

    // Create new AbortController for this request
    abortControllerRef.current = new AbortController();

    setLoading(true);
    setError(null);

    try {
      const response = await fetch(
        `https://api.weatherbit.io/v2.0/current?city=${cityName}&key=${API_KEY}`,
        { signal: abortControllerRef.current.signal }
      );

      if (!response.ok) {
        throw new Error('Failed to fetch weather data');
      }

      const data = await response.json();
      setWeatherData(data.data[0]);
      setLoading(false);
    } catch (err) {
      if (err.name === 'AbortError') {
        console.log('Fetch aborted');
      } else {
        setError(err.message);
        setLoading(false);
      }
    }
  };

  // useEffect with dependency array - refetches when city changes
  useEffect(() => {
    fetchWeather(city);

    // Cleanup function to cancel ongoing requests
    return () => {
      if (abortControllerRef.current) {
        abortControllerRef.current.abort();
      }
    };
  }, [city]); // Dependency array - effect runs when city changes

  const handleSubmit = (e) => {
    e.preventDefault();
    if (inputCity.trim()) {
      setCity(inputCity.trim());
      setInputCity('');
    }
  };
};
```

```

    }
  };

  return (
    <div style={styles.container}>
      <h1 style={styles.title}>Weather Dashboard</h1>

      <form onSubmit={handleSubmit} style={styles.form}>
        <input
          type="text"
          value={inputCity}
          onChange={(e) => setInputCity(e.target.value)}
          placeholder="Enter city name"
          style={styles.input}
        />
        <button type="submit" style={styles.button}>
          Get Weather
        </button>
      </form>

      {loading && <div style={styles.loading}>Loading weather data...</div>}

      {error && <div style={styles.error}>Error: {error}</div>}

      {weatherData && !loading && !error && (
        <div style={styles.weatherCard}>
          <h2 style={styles.cityName}>{weatherData.city_name}</h2>
          <div style={styles.weatherInfo}>
            <p style={styles.temp}>{weatherData.temp}°C</p>
            <p style={styles.description}>
              {weatherData.weather.description}
            </p>
            <div style={styles.details}>
              <p>Humidity: {weatherData.rh}%</p>
              <p>Wind Speed: {weatherData.wind_spd} m/s</p>
              <p>Pressure: {weatherData.pres} mb</p>
              <p>Clouds: {weatherData.clouds}%</p>
            </div>
          </div>
        </div>
      )}
    </div>
  );
};

// Styles object
const styles = {
  container: {
    maxWidth: '600px',
    margin: '0 auto',
    padding: '20px',
    fontFamily: 'Arial, sans-serif'
  },
  title: {
    textAlign: 'center',
    color: '#333',
    marginBottom: '30px'
  },
  form: {
    display: 'flex',
    gap: '10px',
    marginBottom: '30px'
  },
  input: {
    flex: '1',
    padding: '10px',
    fontSize: '16px',
    border: '1px solid #ddd',
    borderRadius: '4px'
  },
  button: {

```

```

padding: '10px 20px',
fontSize: '16px',
backgroundColor: '#007bff',
color: 'white',
border: 'none',
borderRadius: '4px',
cursor: 'pointer',
':hover': {
  backgroundColor: '#0056b3'
}
},
loading: {
  textAlign: 'center',
  color: '#666',
  fontSize: '18px'
},
error: {
  textAlign: 'center',
  color: '#dc3545',
  fontSize: '16px',
  padding: '10px',
  backgroundColor: '#f8d7da',
  borderRadius: '4px'
},
weatherCard: {
  backgroundColor: '#f8f9fa',
  padding: '20px',
  borderRadius: '8px',
  boxShadow: '0 2px 4px rgba(0,0,0,0.1)'
},
cityName: {
  margin: '0 0 15px 0',
  color: '#333',
  fontSize: '24px'
},
weatherInfo: {
  textAlign: 'center'
},
temp: {
  fontSize: '48px',
  fontWeight: 'bold',
  color: '#007bff',
  margin: '10px 0'
},
description: {
  fontSize: '20px',
  color: '#666',
  marginBottom: '20px'
},
details: {
  display: 'grid',
  gridTemplateColumns: 'repeat(2, 1fr)',
  gap: '10px',
  marginTop: '20px',
  textAlign: 'left'
}
};

export default Weather;

```

Key Implementation Features

1. Dependency Array Usage

- The `useEffect` hook includes `[city]` as a dependency array
- Effect runs when component mounts AND whenever the city state changes

- Fetches new weather data automatically when user searches for a new city

2. Cleanup Function Implementation

javascript

```
useEffect(() => {
  fetchWeather(city);

  // Cleanup function
  return () => {
    if (abortControllerRef.current) {
      abortControllerRef.current.abort();
    }
  };
}, [city]);
```

- Uses `AbortController` to cancel ongoing fetch requests
- Cleanup runs before component unmounts or before next effect execution
- Prevents memory leaks and state updates on unmounted components

3. State Management

- `useState` hooks manage city, weather data, loading, error, and input states
- Real-time input handling with controlled component pattern

4. Conditional Rendering

- Loading states display while fetching data
- Error messages shown when API calls fail
- Weather data displayed only when successfully retrieved

Expected Output

When you run this application:

1. **Initial Load:** Fetches and displays weather for default city (London)

2. **User Search:**

- Enter "New York" in input field
- Click "Get Weather" button
- Component fetches and displays NYC weather data

3. **UI Display Shows:**

- City name
- Temperature in Celsius
- Weather description
- Additional details (humidity, wind speed, pressure, cloud coverage)

API Response Example

The Weatherbit API returns data in this format :

json

```
{
  "data": [{
    "city_name": "London",
    "temp": 18.5,
    "weather": {
      "description": "Partly cloudy"
    },
    "rh": 65,
    "wind_spd": 3.2,
    "pres": 1012,
    "clouds": 40
  }]
}
```

To Use This Code

1. Sign up for a free API key at [Weatherbit.io](https://weatherbit.io)
2. Replace YOUR_API_KEY with your actual API key
3. The free tier allows 50 requests per day
4. Import and use the weather component in your app

This implementation fulfills all requirements:

- ✓ Fetches weather data using useEffect
- ✓ Uses dependency array to refetch on city change
- ✓ Implements cleanup with AbortController
- ✓ Includes input field and button for city selection
- ✓ Displays weather information properly